LUTGPT: a rank-mediated transformer backbone via differentiable lookup tables

An existence proof at nanochat scale, a transparent efficiency profile, and a hardware bet

Anatoly Starostin Eugene Izhikevich

Abstract

We start from a working hypothesis that the computation performed inside a spiking neural network can be modelled, in the abstract, as a sequence of mappings between rank/order patterns of input activations and rank/order patterns of output activations [1]. A differentiable lookup-table (LUT) primitive is the natural carrier of such mappings: each table decodes its input by pairwise sign comparisons of selected anchor pairs, emits a learnt row, and trains via a soft surrogate of that discrete decision. A toy LUT transformer was proposed earlier as a proof-of-concept but never tested on real text at real context length and a real vocabulary size.

This report asks whether the LUT family can extend to a model that works at nanochat scale [2]: real ClimbMix text data, a real BPE tokeniser, real context length, and a real vocabulary size. We answer in the affirmative for the *backbone*: the publishable model, LUTGPT, replaces every Q, K, V, output, and residual projection in a six-layer transformer with a multi-head LUT, and matches a vanilla baseline in validation loss at a matched training-token budget on the same data. Two interfaces remain Euclidean and require dense matrix multiplications — scaled-dot-product attention (SDPA) and the linear unembedder; both are open problems we deliberately do not solve here. On current GPU hardware the LUT model is not competitive in wall-clock; we are explicit about this. The bet is on rank-coded sparse-lookup hardware that does not yet exist, for which the LUT family's $\sim$3–4$\times$ lower per-token HBM read on the backbone and bounded active-parameter set would become a decisive advantage.

1 Introduction

1.1 The starting hypothesis

The spiking manifesto [1] sets out a working hypothesis: a spiking neural network, viewed as a computation, is a *permutation mapper*. Its inputs are rank/order patterns over spike arrival times. Its outputs are new rank/order patterns over emitted spikes. What it computes is determined by the ordering of its inputs, not by their absolute magnitudes; how it does so — the biophysics, the ion channels, the dendritic integration — is an implementation detail.

Two consequences follow. First, the right abstraction for studying these networks is the mapping itself, not the dynamics. Second, that mapping is naturally captured by a finite, trainable, discrete object: a lookup table whose decision boundary is rank-coded and whose output rows are learnt by gradient descent on a soft surrogate. The manifesto proposes this LUT abstraction as the primitive of a new family of architectures and demonstrates a small proof-of-concept LUT transformer at short context. The demonstration shows the idea; it does not show whether the family scales.

1.2 The LUT primitive in brief

Here is a brief reminder of the LUT idea. The full derivation is in Section 2; here we only need a working picture.

A LUT layer maps an input vector x to an output vector y in two steps. *(1) Encode by pairwise sign comparisons.* The layer fixes N anchor pairs (a_j, b_j) at initialisation. For each pair it computes one bit $\mathbf{1}[x_{a_j} > x_{b_j}]$, and packs the N bits into a single row index

$$b^\star \;=\; \sum_{j=1}^{N} \mathbf{1}[x_{a_j} > x_{b_j}] \cdot 2^{N-1-j} \;\in\; \{0, 1, \ldots, 2^N - 1\}.$$

The encoding is purely rank-coded: each bit depends only on the order of two coordinates of x, not on the magnitude of their difference. *(2) Decode by row gather.* The layer holds a learned weight table $W \in \mathbb{R}^{2^N \times D_{\text{out}}}$ and outputs the gathered row $y = W[b^\star] \in \mathbb{R}^{D_{\text{out}}}$. The decoding is an arbitrary learned mapping from the discrete row index to a real output vector.

A *multi-head LUT* runs many such tables in parallel. The layer is parameterised by an input width E, a per-head output width D_{out}, a head count H, and a tables-per-head tph. It holds $H \cdot \text{tph}$ independent tables, each with its own anchor pairs (a_j, b_j) and its own weight table W. The per-head output is the sum of the tph tables' outputs in that head. Training is end-to-end through a soft surrogate of the discrete row-selection step, derived in Section 2.

1.3 Our research question

The question this work addresses is concrete:

> *Can the toy LUT transformer extend to a real model that handles real text data at real context length and a real vocabulary size, with validation loss comparable to a vanilla transformer baseline on the same data?*

From nanochat [2] we adopt the dataset (ClimbMix), the BPE tokeniser, its 32,768-token vocabulary, and a 512-token context length. The baseline we compare against is a vanilla, RoPE-augmented transformer with a separate unembedder — the final linear projection to logits has its own weight matrix, distinct from the token embedding — that we constructed ourselves on the same data — deliberately simpler than nanochat's own GPT, because we want a clean LUT-vs-non-LUT comparison rather than one that mixes the LUT axis with more sophisticated baseline-side techniques. Even so, vanilla + RoPE is a strong reference; nanochat is small enough to iterate on, large enough that matching it on validation loss is informative. The pragmatic version of the claim: *if the LUT family solves nanochat, no fundamental wall has appeared at this scale; further scaling is then an engineering question, not an existence question.*

1.4 The model we present (LUTGPT)

LUTGPT is the model that resulted: a six-layer transformer where every attention projection (Q, K, V, output) and every residual contribution is a multi-head LUT, while scaled-dot-product attention itself and the linear unembedder remain standard. This is a hybrid model, not a matmul-free one, and we are explicit about that. Our experiments did not produce a viable LUT-based replacement for either softmax attention or the final linear classifier; we treat both as open problems.

Two forward modes are published with the same trained backbone:

- **Hard mode.** Each LUT performs a sign-pack lookup: pure pairwise comparisons of anchor pairs. The backbone is *magnitude-blind*: a layer can tell which anchor coordinate is larger, never by how much. No multiplications are performed inside the LUT itself.
- **Hybrid-smooth mode.** The forward blends the main row with a single Hamming-1 neighbour at the least-confident anchor, with a soft weight derived from the magnitude of that anchor's

difference. This costs one extra row read and a scalar blend per LUT, in exchange for a decent loss improvement.

Both modes share the same soft-surrogate backward and the same trained weights. We present both because they show the cost of magnitude information: hybrid-smooth is better in loss, hard is closer to the manifesto's purity claim, and the gap is small enough that the rank-only backbone is a legitimate operating point.

1.5 Practical efficiency

The publishable model is not competitive on GPUs. Training is slower: every backward pass through a LUT executes a dense matmul-shaped operation against the full K-row neighbourhood (see Section 2). Inference is slower: sparse rank-coded lookups do not use tensor cores, and the model is HBM-fetch-bound on hardware that is optimised for dense matmuls. We are explicit about this.

What is interesting — and what justifies the report — are two architectural properties that are invisible to GPUs but would be central on hardware that can exploit them. The LUTGPT trunk at full backbone width reads $\sim$3$\times$ less HBM per token than the matched vanilla baseline despite holding $\sim$7.7$\times$ more parameters; the narrow-backbone variant reads $\sim$3.8$\times$ less per token at $\sim$4.9$\times$ the parameter count. In both cases the compression density (parameters per byte fetched) on the backbone is roughly twenty times higher than vanilla. And the active-parameter set per token is a bounded sparse subset, predictable at compile time, not the whole weight matrix. On memory-bound or sparsity-native hardware — a class of accelerator we do not yet have but consider plausible to build — these properties translate to a real efficiency win. *We are not claiming such hardware exists; we are presenting the architectural property that would make it matter if it did.*

1.6 Organisation

Section 2 introduces the `MultiHeadLut` primitive and states precisely how its math differs from the manifesto's proposal — two differences are load-bearing, and we identify them explicitly. Section 3 describes the LUTGPT architecture and the dual residual-stream split, including the methodological commitment to baseline matching. Section 4 states the training recipe. Section 5 reports the efficiency analysis and the hardware bet. Section 6 reports the experiments: the vanilla baseline, the full-width LUTGPT result, a narrow-backbone LUTGPT result, and the soft-to-hard gap. Section 7 lists the open problems — LUT-based attention and a LUT-based unembedder are the residual matmul interfaces in our model, and resolving either would be informative independently of the other.

2 The LUT primitive: MultiHeadLut

2.1 Setup

`MultiHeadLut` is a layer

$$F : \mathbb{R}^E \;\to\; \mathbb{R}^{H \times D_{\text{out}}}, \qquad x \mapsto y_h = \sum_{s=1}^{\text{tph}} \text{Lut}_{h,s}(x),$$

parameterised by an input width E, an output width D_{out} per head, a number of heads H, a number of tables per head tph, and a number of anchor pairs (NAP) per table. The layer holds $H \cdot \text{tph}$ independent lookup tables; each one maps x to a D_{out}-vector, and the per-head output sums the tph tables in that head. Two scalar temperatures shared by all tables, stored in log-space and exponentiated on use: T_{soft} (per-anchor sign sharpness) and T_{sel} (row-selection sharpness).

The math below describes a single table $\mathrm{Lut}_{h,s}$ — every formula runs in parallel on all $H \cdot \mathrm{tph}$ of them with their own (a_j, b_j) and W but shared $T_{\mathrm{soft}}, T_{\mathrm{sel}}$.

Anchor pairs and weight table. Each table fixes N anchor pairs $(a_j, b_j)_{j=1}^{N}$ once at module initialisation and never moves them. The pair differences are

$$d_j \;=\; x_{a_j} - x_{b_j}, \qquad j = 1, \ldots, N.$$

The map $x \mapsto (d_1, \ldots, d_N)$ is linear and sparse: each d_j depends only on x_{a_j} and x_{b_j}. A weight table $W \in \mathbb{R}^{K \times D_{\mathrm{out}}}$ with $K = 2^N$ rows is indexed by $b \in \{0,1\}^N$. Bit parity:

$$\chi_j(b) \;:=\; 2b_j - 1 \in \{-1, +1\}.$$

Per-anchor soft sign and row score. For each anchor pair j and each row b define

$$p_j \;=\; \frac{d_j}{T_{\mathrm{soft}} + |d_j|} \;=\; \mathrm{sign}(d_j)\,\frac{|d_j|}{T_{\mathrm{soft}} + |d_j|} \;\in\; (-1, +1), \qquad \mathcal{S}(b) \;=\; \sum_{j=1}^{N} p_j\,\chi_j(b),$$

and the softmax distribution over rows

$$z(b) \;=\; \mathcal{S}(b)/T_{\mathrm{sel}}, \qquad \pi(b) \;=\; \mathrm{softmax}_b\, z(b).$$

p_j is the smooth sign; π is the full-K softmax used by the backward surrogate (Section 2.3).

Main row. The argmax of $\mathcal{S}$ is the hard sign-pack:

$$b^\star \;=\; \arg\max_b \mathcal{S}(b), \qquad b^\star_j \;=\; [d_j > 0].$$

Computing $b^\star$ takes N comparisons and packs the bits into one integer in $\{0, \ldots, K-1\}$.

2.2 Two forward modes

Hard. The published default. Output is the single row picked by the sign-pack:

$$\boxed{\; y^{\mathrm{hard}} \;=\; W[b^\star]. \;}$$

Cost: $O(N)$ to compute $b^\star$, one row gather. No softmax, no row blend; temperatures play no role in y^{hard}.

Hybrid-smooth. An exact two-row softmax restricted to $b^\star$ and its least-confident Hamming-1 neighbour. Let

$$j^\star \;=\; \arg\min_j |d_j|, \qquad b_{\mathrm{alt}} \;=\; b^\star \oplus e_{j^\star}.$$

Any single-bit flip of $b^\star$ yields a row whose score differs from $\mathcal{S}(b^\star)$ by exactly $2|p_j|$. The smallest such gap is at $j = j^\star$, so b_{alt} is the second-best row under $\mathcal{S}$.
Setting $d_{\min} := |d_{j^\star}|$, the exact score gap is

$$\Delta(b^\star, b_{\mathrm{alt}}) \;=\; \mathcal{S}(b^\star) - \mathcal{S}(b_{\mathrm{alt}}) \;=\; 2|p_{j^\star}| \;=\; \frac{2\,d_{\min}}{T_{\mathrm{soft}} + d_{\min}},$$

and the softmax of z restricted to $\{b^\star, b_{\mathrm{alt}}\}$ places mass

$$u \;=\; \sigma(-\Delta/T_{\mathrm{sel}}) \;=\; \frac{1}{1 + \exp(\Delta/T_{\mathrm{sel}})} \;\in\; (0, \tfrac{1}{2}]$$

on the alt row. The output is the two-row blend:

$$\boxed{\; y^{\mathrm{hyb}} \;=\; (1-u)\,W[b^\star] \;+\; u\,W[b_{\mathrm{alt}}]. \;}$$

Both temperatures enter u: T_{soft} shapes Δ through $p_{j^\star}$; T_{sel} rescales the resulting log-odds. Cost: $O(N)$ plus two row gathers and one blend; no materialisation of $\pi \in \mathbb{R}^K$.

Limit. As $|d_{j^\star}|/T_{\text{soft}} \to \infty$ or $T_{\text{sel}} \to 0^+$, $u \to 0$ and $y^{\text{hyb}} \to y^{\text{hard}}$. Hybrid-smooth is a strict refinement of hard: same $b^\star$, extra mass u on the second-best row at the least-confident anchor.

2.3 Soft surrogate backward

Let $g := \partial L/\partial y \in \mathbb{R}^{D_{\text{out}}}$. The backward decomposes into two pieces:

(A) *Weight gradient* flows *only* through the row(s) that actually contributed to y — one row in hard mode, two rows in hybrid-smooth mode.
(B) *Input and temperature gradients* are computed as the analytic gradient of the full-K soft forward $y_{\text{full}} = \sum_b \pi(b)\, W[b]$ from Section 2.1, ignoring the truncation the actual forward applied. Identical in both forward modes.

(A) Weight gradient (mode-dependent)

Hard mode. Differentiating $y^{\text{hard}} = W[b^\star]$ at fixed $b^\star$:

$$\frac{\partial L}{\partial W[r]}^{(\text{hard})} = g\,\mathbf{1}[r = b^\star]; \qquad \text{all other rows } 0.$$

A one-row scatter into W per sample.

Hybrid-smooth mode. Differentiating $y^{\text{hyb}} = (1-u)W[b^\star] + u\,W[b_{\text{alt}}]$ at fixed $u, b^\star, b_{\text{alt}}$:

$$\frac{\partial L}{\partial W[r]}^{(\text{hyb})} = (1-u)\,g\,\mathbf{1}[r = b^\star] \;+\; u\,g\,\mathbf{1}[r = b_{\text{alt}}]; \qquad \text{all other rows } 0.$$

A two-row scatter per sample: weight $(1-u_b)\,g_b$ at row $b_b^\star$, weight $u_b\,g_b$ at row $b_{\text{alt},b}$.

In both cases this is the honest derivative of the truncated forward — no soft attribution to rows the forward did not touch. As $u \to 0$ the hybrid-smooth weight grad collapses to the hard weight grad, consistent with the forward limit.

(B) Input and temperature gradients via the full-soft surrogate

The forward used only $\{b^\star\}$ or $\{b^\star, b_{\text{alt}}\}$, but for $\partial L/\partial x$ and $\partial L/\partial T$ we want a denser local signal that reflects the full neighbourhood of $b^\star$. So we backpropagate as if the forward had been the full-K soft mode of Section 2.1, $y_{\text{full}} = \sum_b \pi(b)\, W[b]$, reusing the same $p_j, \mathcal{S}(b), z(b), \pi(b)$ defined there.

Through the softmax.

$$\begin{aligned}
g_\pi(b) &= g^\top W[b] && \in \mathbb{R}^K,\\
\langle g_\pi, \pi\rangle &= \sum_b g_\pi(b)\,\pi(b), &&\\
g_z(b) &= \pi(b)\,\big(g_\pi(b) - \langle g_\pi, \pi\rangle\big) && \text{(softmax Jacobian)},\\
g_{\mathcal{S}}(b) &= g_z(b)/T_{\text{sel}}. &&
\end{aligned}$$

Through the row-score sum. $\mathcal{S}$ is linear in p_j with coefficient $\chi_j(\cdot)$, so

$$g_{p_j} = \sum_b g_{\mathcal{S}}(b)\,\chi_j(b).$$

Through the soft sign. $p_j = d_j/(T_{\text{soft}} + |d_j|)$, so differentiating directly,

$$\frac{\partial p_j}{\partial d_j} \;=\; \frac{T_{\text{soft}}}{(T_{\text{soft}} + |d_j|)^2}, \qquad g_{d_j} \;=\; g_{p_j}\,\frac{T_{\text{soft}}}{(T_{\text{soft}} + |d_j|)^2}.$$

Through the anchor-pair difference operator. Since $d_j = x_{a_j} - x_{b_j}$ is a sparse linear map, the input gradient is a scatter-add at the anchor positions:

$$(\partial L/\partial x)_{a_j} \;\mathrel{+}=\; g_{d_j}, \qquad (\partial L/\partial x)_{b_j} \;\mathrel{-}=\; g_{d_j}.$$

Temperatures (log-parameterised, $\partial/\partial \log T \;=\; T \cdot \partial/\partial T$). z depends on T_{sel} only via the rescale $z = \mathcal{S}/T_{\text{sel}}$, and on T_{soft} only via p_j (hence through d_j in the same way as x):

$$\boxed{\frac{\partial L}{\partial \log T_{\text{sel}}} \;=\; -\sum_b g_z(b)\, z(b), \qquad \frac{\partial L}{\partial \log T_{\text{soft}}} \;=\; -\sum_j g_{d_j}\, d_j.}$$

The temperature gradients are nonzero even in hard forward mode, where the temperatures had no effect on y at all — the surrogate gives them an update through the soft-pipeline Jacobian.

2.4 Why the asymmetry

Weight grad is hard because the forward is hard. The forward only ever reads $W[b^\star]$ (hard) or $\{W[b^\star], W[b_{\text{alt}}]\}$ (hybrid-smooth); any soft attribution to the remaining $K-1$ or $K-2$ rows would be a phantom update with no matching forward contribution. Restricting the weight grad to the rows the forward actually touched keeps the gradient an unbiased derivative of the actual computation.

Input/temperature grad is soft because we want a richer signal. x enters the forward only through $b^\star$ (hard) or through $b^\star$, b_{alt}, and u (hybrid-smooth). The honest derivative w.r.t. x is therefore either identically zero (hard: $b^\star$ is a discrete function of x, the gather is x-independent at the chosen index) or dominated by a single coordinate (hybrid-smooth: only the $j^\star$-th anchor pair feeds u). Neither is a usable training signal. The full-soft surrogate restores a dense signal at all N anchor pairs and gives the temperatures a useful update each step, at the cost of not being the literal Jacobian of the truncated forward. Empirically this trade — honest weight update, smooth input update — is what makes both forward modes trainable end-to-end while leaving inference as a sign-pack lookup.

2.5 Two material differences from the manifesto's math

Our LUT math is close to the abstraction proposed in [1], but two specific differences are load-bearing for whether the LUT chain trains at scale:

(1) Top-2 "blended alternative" as an implemented forward. The manifesto suggested in passing that one could blend the chosen row with alternative rows during training; the suggestion was not implemented in the toy LUT transformer. We implement the simplest non-trivial version (one Hamming-1 alternative at the least-confident anchor pair) and find it trains end-to-end. The implementation is small but specific: it introduces a soft mixing weight derived from the magnitude of the least-confident anchor difference, an asymmetric two-row weight gradient, and a parameterised tradeoff (via T_{sel}) between sharpening to the hard forward and spreading the gradient across alternatives.

(2) Full K-row backward as a necessary condition. The soft surrogate above scores *all* K rows, and the gradient $\partial L/\partial x$ flows through every one of them, including rows the forward never visited. A naive straight-through alternative — backward that only routes gradient through the chosen row's weights, ignoring the other $K-1$ rows — still trains, but converges to substantially worse validation loss at nanochat scale; switching from the 1-row backward to the full-K backward visibly lowers the loss curve in our experiments. This is not a polishing observation: it is a quality problem, not a numerical instability, and we have reproduced it across many recipes. We therefore treat the full-K backward as the binding empirical requirement of the family, not an optimisation choice.

2.6 Input normalisation (MeanAbsNorm)

Each LUT input is normalised before the anchor-pair differences are computed:

$$\hat{x} \;=\; \frac{x}{\mathrm{mean}(|x|) + \varepsilon},$$

with the mean taken over the input dimension. The hard forward is scale-invariant per LUT — the sign-pack only asks $x_{a_j} > x_{b_j}$, which survives any positive rescaling of x — so removing this step does *not* change which row a given input gathers, and the rank-decoding interpretation of the forward holds without it.

The reason `MeanAbsNorm` is necessary is the soft surrogate backward. The soft sign $p_j = d_j/(T_{\text{soft}} + |d_j|)$ is *not* scale-invariant: T_{soft} has a fixed value, calibrated to a specific magnitude regime for the anchor-pair differences d_j. If $|x|$ drifts during training (the residual stream growing or shrinking layer-by-layer, for instance), $|d_j|$ drifts with it, and T_{soft} moves out of its calibrated point. Two failure modes follow. If $|d_j| \ll T_{\text{soft}}$, all $|p_j|$ collapse toward 0, the row score $\mathcal{S}$ is flat, the surrogate softmax distributes mass roughly uniformly over K rows, and the gradient signal toward the chosen row vanishes. If $|d_j| \gg T_{\text{soft}}$, all $|p_j|$ saturate at 1, the surrogate collapses onto the single row $b^\star$, and the gradient at non-chosen rows vanishes. Either way, the soft surrogate stops providing the dense local signal it was designed to provide.

`MeanAbsNorm` pins $\mathrm{mean}(|x|) \approx 1$, which holds $|d_j|$ in the regime T_{soft} is set for, which keeps the soft surrogate well-conditioned for the entire run. It is a training-time stabiliser for the soft-backward gradient signal, not a precondition for the rank-decoding interpretation of the forward.

2.7 Anchor pairs as fixed structural prior

The anchor pairs (a_j, b_j) are sampled once at module construction time from a balanced canonical-coverage policy (defined below) and never moved. They are a structural prior, not learnt content. What is learnt is the table content $W \in \mathbb{R}^{K \times D_{\text{out}}}$ and the two scalar temperatures $T_{\text{soft}}, T_{\text{sel}}$. This is a deliberate choice: making the anchor pairs trainable introduces a non-differentiable selection problem (which input coordinate to point at) without clear empirical benefit, while fixing them gives the layer a known, decomposable structure.

Balanced canonical-coverage policy. Let $\mathcal{P}$ denote the canonical pool of all ordered coordinate pairs (a, b) with $a < b$, so $|\mathcal{P}| = \binom{E}{2}$. For each head independently, we need to allocate N anchor pairs to each of its tph tables — a total of $\mathrm{tph} \cdot N$ slots per head. We fill these slots by concatenating one or more random permutations of $\mathcal{P}$ and truncating to length $\mathrm{tph} \cdot N$, then reshaping into tph rows of N entries each.

This procedure has three properties:

- *Coverage.* If $\mathrm{tph} \cdot N \geq |\mathcal{P}|$, every canonical pair appears at least once across that head's tables. At the LUTGPT (exp750) configuration the canonical pool size is $\binom{384}{2} = 73{,}536$; per-head slot

counts (e.g. $\text{tph} \cdot N = 256 \cdot 6 = 1{,}536$ for v_lut, $512 \cdot 7 = 3{,}584$ for out_proj) are well below the pool size, so each LUT samples a small balanced subset of the canonical pool rather than achieving full coverage. Each pair has equal probability of appearing.

- *Balance.* Pairs are distributed across tables by a random permutation, so there is no clustering of related pairs in a single table and no preferred coordinate. Each pair appears in at most $\lceil (\text{tph} \cdot N)/|\mathcal{P}| \rceil$ tables of the head.
- *Within-table distinctness.* The N anchor pairs of any single table are distinct (no LUT has two anchor pairs comparing the same two coordinates). Duplicates that can appear at random-permutation tile boundaries are repaired in place by swapping with a non-duplicate slot in another table of the same head.

The only nondeterministic element of the architecture is the seed used to draw the permutations; once the seed is fixed, every anchor pair in the model is determined.

3 The LUTGPT architecture

This section is structured as follows: Figure 1 shows the vanilla baseline we compare against, Figure 2 shows LUTGPT. The text that follows explains what is matched between them and why each LUTGPT-specific architectural choice is the way it is.

3.1 The vanilla baseline

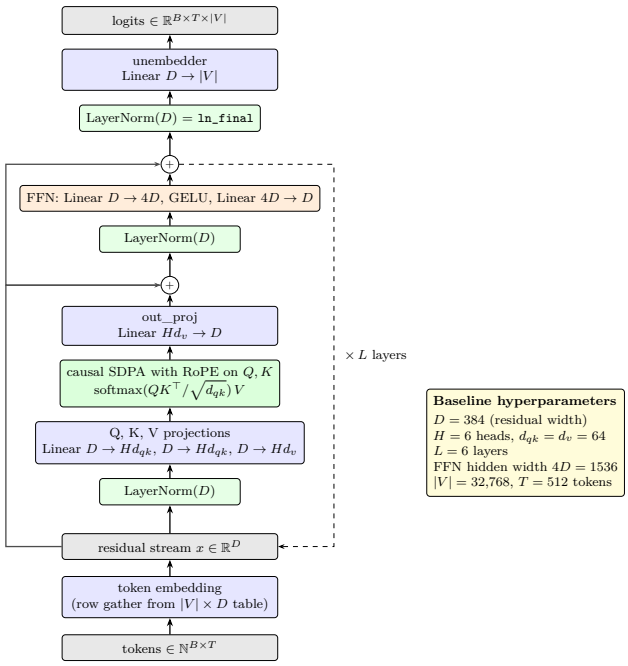

Figure 1: Vanilla baseline transformer. One residual stream of width D. Each of L layers contains a multi-head attention sub-block and a feed-forward network sub-block, each with its own LayerNorm and residual connection.

The baseline is a vanilla, RoPE-augmented transformer [3, 4] with a separate unembedder (its own weight matrix, distinct from the token embedding), constructed by us as a deliberately simple comparison point on the nanochat data [2]. Nanochat itself proposes a more sophisticated GPT

variant with several modern techniques; using that as the baseline would conflate the LUT-vs-non-LUT axis with the simple-vs-sophisticated-non-LUT axis. Vanilla-plus-RoPE is the simplest well-understood reference — strong enough that matching it is a real signal, clean enough that the comparison isolates the LUT-vs-non-LUT axis from architectural improvements orthogonal to it.

3.2 LUTGPT

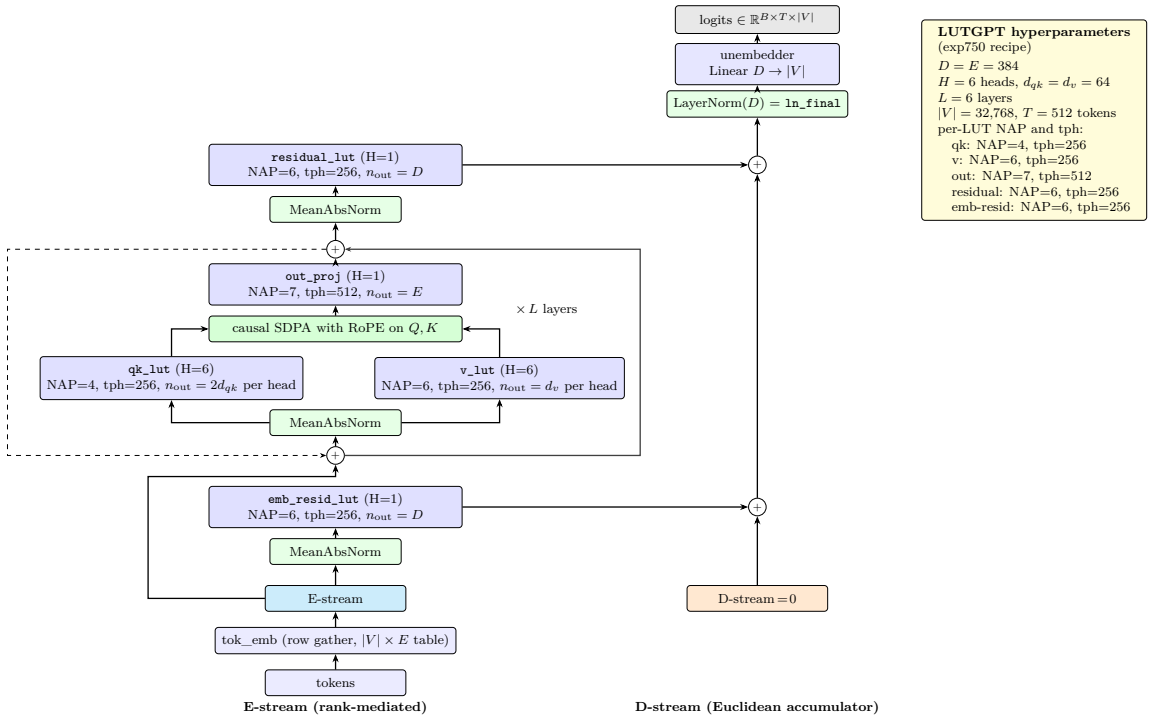

Figure 2: LUTGPT (exp750 recipe). Two residual streams: the E-stream (centre, width E) is the rank-mediated backbone, mutated only by the per-layer `out_proj`; the D-stream (right, width D) is the Euclidean accumulator, written by `emb_resid_lut` once and by `residual_lut` once per layer, read by the linear unembedder. There is no FFN. The two streams are at the same width ($E = D = 384$); the split is structural, not size-based — see Section 3.4.

3.3 Methodological commitment: baseline matching

The two architectures share every hyperparameter that sits at a fair-comparison interface:

- $D = 384$ — the residual stream width on the side feeding the linear unembedder. Matched so our unembedder operates at the same width as vanilla's.
- $d_{qk} = 64$ per head — Q/K dimension at SDPA. Matched so the attention machinery sees the same shape of Q, K.
- Vocab size 32,768 and sequence length 512 — inherited from nanochat. $H = 6$ heads and $L = 6$ layers — our choices, picked at the typical small-GPT scale for fast iteration, and matched on both sides.
- Separate embedding and unembedding matrices on both sides; ClimbMix data shards, BPE tokeniser, RoPE positional encoding, AdamW for non-LUT parameters — inherited unchanged.

The LUTGPT-specific dimensions ($E = 384$, $d_v = 64$, the per-LUT NAP and tph) are then chosen freely, and motivated in Section 3.7 below.

3.4 Dual residual stream

The intuition for the dual stream is: keep the vanilla residual stream as-is, and put a different backbone next to it. A vanilla transformer maintains one residual stream of width D; the backbone is the L-layer stack of attention + FFN blocks, and after each layer the backbone adds an update to the residual stream. The classifier reads the residual stream at the top. LUTGPT keeps exactly this picture, but swaps the backbone.

- **D-stream** (width $D = 384$): the standard residual stream. Same role as vanilla's residual stream and the same width. It is initialised at the embedding interface by `emb_resid_lut`, gets a contribution from each layer (added by `residual_lut`), and is read at the top by `ln_final` and the linear unembedder, which produce the logits. From the classifier's point of view, LUTGPT looks like a vanilla transformer.
- **E-stream** (width $E = 384$): the rank-coded backbone that produces the per-layer contributions. Where vanilla's backbone is dense matmuls (Q/K/V projections, attention, FFN), LUTGPT's backbone is a rank-mediated computation graph of LUTs. The E-stream is the value the backbone passes between its own layers; every consumer of the E-stream is a LUT preceded by `MeanAbsNorm`, so the information that propagates along E is rank-coded (only the order of coordinates matters to the next LUT). At each layer, the backbone exports a contribution into the D-stream via `residual_lut`.

The split is not for capacity. The two streams have the same dimension. The split exists because the backbone (which wants to operate on ranks) and the classifier (which wants Euclidean magnitudes) want different kinds of representation, and a single residual stream would have to satisfy both simultaneously. Keeping them separate lets each be exactly what it needs to be.

3.5 Three roles for LUTs at the interfaces

Every LUT in LUTGPT reads from the E-stream and emits a row from its weight table, but what *happens* to that row downstream depends on the LUT's role. Four distinct interface patterns appear:

- **`qk_lut` → SDPA dot product.** The dot product $Q \cdot K^\top$ is the one operation in the model that strictly requires real-valued magnitudes; there is no rank-only way to compute a similarity that respects scale. The Q and K LUTs are accordingly the *adapters* from the rank-mediated E-stream into a real-valued representation that SDPA can read. RoPE is applied on top.
- **`v_lut` → attention convex sum → `out_proj`.** The V LUT is structurally different. Its output is multiplied by the soft attention weights, which are non-negative and sum to 1, giving a convex combination of V rows that then flows into the next LUT in line. This convex mixing is the *only* operation in the model that takes LUT outputs and recombines them *before* a downstream LUT re-decodes them; the V LUT must be trained so its outputs remain mixable, i.e., convex combinations of plausible V rows stay inside a rank-decodable region for `out_proj`. The V LUT is rank-coded in input, rank-coded (via mixability) in output.
- **`out_proj` → E-stream → next layer's LUTs.** `out_proj` is an ordinary LUT in this scheme — not a reverse adapter. Its input is the SDPA output, which by the V-LUT mixability argument above is a convex combination of rank-decodable rows and therefore still lies in the

rank-decodable region. `out_proj` reads it with sign-pack like any other LUT, and its row gather is added to the E-stream as the layer's attention contribution.

- **`emb_resid_lut`, `residual_lut` → D-stream → linear unembedder.** `emb_resid_lut` (once, at the embedding interface) and `residual_lut` (once per layer) write into the D-stream, which is read by the linear unembedder. Both are adapters from a rank-coded E-stream input to a real-valued contribution to the D-stream, in the same family as Q/K.

3.6 No separate FFN

The vanilla layer in Figure 1 carries two computational blocks: an attention sub-block (Q/K/V projections, attention, output projection) and an FFN afterward, with the FFN typically holding the bulk of the per-layer parameters. LUTGPT (Figure 2) has no separate FFN. The `out_proj` LUT absorbs that role.

In vanilla, `out_proj` is a thin linear projection from concatenated attention heads to the residual width, and the FFN that follows is where most of the per-layer expressivity lives. In LUTGPT, `out_proj` is itself a multi-head LUT: it reads the post-attention representation, decodes it through anchor-pair sign comparisons, and gathers from a large learnt weight table to produce the residual update. The expressivity that the vanilla FFN provides through wide hidden linear layers and a nonlinearity is provided here by the LUT lookup mechanism itself — a learned discrete-indexed function with $K \cdot D_{\text{out}}$ degrees of freedom per table, capturing nonlinearity through the row gather. Per-layer evidence accumulation into the classifier path is then a separate role, taken by `residual_lut` writing into the D-stream (a role that does not exist in the vanilla architecture, since vanilla shares a single residual stream for both the backbone and the classifier input).

3.7 LUT-specific hyperparameter choices

The dimensions matched against vanilla (D, d_{qk}, H, vocab, context length) are fixed by the baseline-matching commitment of Section 3.3. The remaining choices are LUT-specific:

Backbone width E and per-head dimensions d_{qk}, d_v. The constraint $E = H \cdot d_v$ is always enforced so the SDPA output (concatenation of H heads of d_v values) lands at the E-stream width with no reshape and `out_proj` reads an E-dimensional input. We report two configurations (Section 6):

- **Full-width backbone** (`exp754`): $d_v = d_{qk} = 64$ and $E = D = 384$. Per-head Q, K, and V dimensions all match the vanilla baseline; the backbone runs at the same width as the D-stream and the unembedder readout.
- **Narrow backbone** (`exp755`): $d_v = 32$ and $E = 192$, with $d_{qk} = 64$ held to the baseline. The LUT-bearing backbone is half-width while the D-stream stays at $D = 384$ for the unembedder readout.

These two configurations together test the asymmetric-architecture claim of Section 3.4: the rank-coded stream where the LUTs compute is allowed to be narrow, while the Euclidean accumulator stays wide for the unembedder. The remaining LUT-specific choices (anchor pairs per table, tph, qk-lut sharing) are identical across both configurations.

Anchor pairs per table (NAP). Larger NAP gives the LUT more discrimination capacity (each table has $K = 2^{\text{NAP}}$ rows), at quadratic cost in the soft surrogate backward (a full K-way softmax per (sample, table)). We use NAP $\in \{4, 6, 7\}$ in LUTGPT, scaled to the role of each LUT:

- NAP $= 4$ ($K = 16$) for `qk_lut`. The qk_lut feeds the SDPA dot product. Its job is to make Q and K discriminative enough that softmax-attention picks the right positions; the dot-product

score and the downstream attention weights then carry the fine-grained signal.

The smaller NAP here is also a plasticity choice. Attention patterns emerge during training and keep changing throughout the run, so the Q/K projections must stay flexible end-to-end. In a vanilla transformer Q and K come from dense linear projections, and every backward step updates every weight; gradients are uniformly strong. In LUTGPT each token-level lookup touches only one row of a $K \times D_{\text{out}}$ table (two in hybrid-smooth), so the per-row gradient sees only a fraction $\sim B/K$ of the batch per step. Larger K makes the per-row update count smaller and trainability deteriorates — at the SDPA interface, where the layer has to relearn its own attention pattern continuously, this is the most painful place for that to happen. Smaller K keeps every Q/K row in the active-training regime; $K = 16$ is the smallest value at which the family still has enough distinct Q/K vectors to express useful attention patterns in our experiments.

- NAP = 6 ($K = 64$) for `v_lut`, `residual_lut`, and `emb_resid_lut`. Wider tables at the streams where the LUT output directly populates a per-token vector that downstream layers (or the unembedder) read at full width.
- NAP = 7 ($K = 128$) for `out_proj`. Out-proj absorbs the FFN's role (Section 3.6) and is the single LUT in the layer that has to carry the bulk of the per-layer transformation; we give it the largest row count.

Tables per head tph**.** Each LUT's per-head output is the sum of its tph table outputs (Section 2.1). Larger tph gives the layer more capacity at linear cost in the forward and the soft backward. We use tph = 256 for the qk, v, residual, and emb-resid LUTs (a common default), and tph = 512 for `out_proj` specifically, because `out_proj` absorbs the FFN's role and is the single LUT in the layer carrying the bulk of the per-layer expressivity.

qk_lut sharing for Q and K. A single LUT generates both Q and K projections by emitting an output of width $2d_{qk}$ per head, then splitting the output into Q and K inside the layer. This is a small economy: Q and K are needed at the same time, at the same per-head width, and from the same input, so they share the lookup overhead.

4 Training recipe

LUT weight tables are trained with Lion [5]; all non-LUT parameters — the token embedding, the unembedder, the layer norms, the per-LUT temperatures — are trained with AdamW [6]. The choice is a memory choice, not a quality choice. Lion stores one momentum buffer per parameter; AdamW stores two (m and v). For a model that has roughly 5–8× as many parameters as the matched vanilla baseline (depending on backbone width), the optimiser state on LUT weights is a meaningful fraction of training memory. Empirically, Lion matches AdamW on LUT-weight quality — comparable validation loss, never worse on our recipes — at half the per-parameter state. We use Lion for the LUT tables because of this memory saving, and AdamW for non-LUT parameters because that is the conventional default and we do not benefit from changing it.

The default schedule is a single cosine decay over the full 16,000-step run with a 10% linear warmup, peaking at $\text{lr}_{\text{adam}} = 3 \cdot 10^{-4}$ for AdamW (non-LUT parameters) and $\text{lr}_{\text{lion}} = 2 \cdot 10^{-4}$ for Lion (LUT weights), both decaying to 0.1× peak. Effective batch is held constant across the run at $48 \cdot 512 = 24{,}576$ tokens per step; we do not use a curriculum on batch size. Gradient clipping is `clip_grad_norm` to 1.0 over all trainable parameters.

The forward mode is held fixed within a single run. Whether to train in hybrid-smooth (smaller training loss, larger soft-to-hard eval gap) or in hard (larger training loss, deployable as-is on a magnitude-blind hard forward) is a model choice; we report both in Section 6.

5 Efficiency analysis and the hardware bet

5.1 GPU wall-clock cost

LUTGPT is slower than the matched vanilla baseline on training and on inference, on every GPU we measured. Reasons:

- The model has 5–8× more parameters than the matched vanilla baseline (7.7× at the full-width backbone, 4.9× at the narrow backbone); even at proportionally lower fetch-bandwidth-per-token, the absolute compute is greater.
- The LUT backward path executes a soft-surrogate matmul (sparse selection mass times grad output) for every layer in every step; this is dense in compute even though sparse in selection.
- LUT forward gathers do not use tensor cores; on consumer and datacenter GPUs the model is HBM-bandwidth-bound on the LUT layers while the SDPA layers happily use bf16 tensor cores. The mix is not what current GPUs are tuned for.

We report wall-clock numbers as found and do not promise improvements achievable on this hardware class.

5.2 Bandwidth and active-parameter profile

The properties that justify presenting the family despite the GPU result are two:

- **HBM read per token, trunk.** A LUTGPT layer reads $H \cdot \text{tph} \cdot D_{\text{out}} \cdot \text{bytes}$ for the gathers — one row per table per token. Summed across all LUTs in the trunk and across $L = 6$ layers, the full-width backbone (`exp754`) reads $\sim 7.3\,\text{MB}$ per token, against $\sim 21.2\,\text{MB}$ for the vanilla matmul backbone at $D = 384$ — a factor of ~ 2.9. The narrow backbone (`exp755`) reads $\sim 5.5\,\text{MB}$ per token — a factor of ~ 3.8. The factor scales further at larger $K = 2^N$: bandwidth is independent of K, while vanilla's projection cost scales with output width.
- **Active parameters per token.** A vanilla projection activates every weight per token. A LUTGPT projection activates exactly $H \cdot \text{tph}$ rows per token — a small, predictable subset of $K \cdot H \cdot \text{tph}$ total. The active-parameter fraction is below 5% per token in both configurations, and the active set is decided by sign comparisons, not by data-dependent computation.

Compression density. To make the bandwidth picture explicit, define

$$\text{compression density} \;=\; \frac{\text{stored parameters}}{\text{HBM bytes fetched per token}}.$$

This measures how much model capacity a layer holds per unit of memory traffic, and is the relevant metric on bandwidth-bound hardware. For a dense linear layer every weight is fetched exactly once per token, so the compression density is fixed at $1/b$ params per byte (where b is the number of bytes per stored value, e.g. $b = 2$ at bf16) — a ceiling that no choice of width can raise. For a multi-head LUT each table holds K rows but only one row per (table, token) is fetched at inference, so the compression density is K/b params per byte: the table holds $K\times$ more weights than a dense layer would at the same per-token bandwidth budget. Doubling K doubles the stored parameters of a LUT but does not change its per-token bytes fetched. Aggregated across the LUTGPT trunk against vanilla's matmul backbone, the compression density is roughly 20× higher in both configurations of Section 6.

On hardware that fetches only the active rows from a compressed indexed store — the class of accelerator we have in mind — the parameter count stops being a cost. The dense linear

unembedder (a $D \times |V|$ matmul, identical to vanilla and accounting for the majority of total per-token HBM read in either model) is the dominant remaining matmul interface; we leave it as one of the open problems in Section 7.

5.3 The bet

The bet is straightforward: *the LUT family is presented as an architecture that maps cleanly onto rank-coded sparse-lookup hardware, and we conjecture that hardware of this kind would make the family competitive or preferred at scale.* We are not claiming such hardware exists. We are presenting the architectural properties that would make it matter if it did — a low HBM read per token and a sparse, predictable active-parameter set — and arguing that they justify holding the LUT family open as a research direction even when current GPUs make it look uneconomical.

6 Experiments

6.1 Data, budget, and hardware

All runs in this section use the optimiser and learning-rate recipe of Section 4 (AdamW for non-LUT parameters; Lion for LUT weights; warmup-cosine schedule; `clip_grad_norm` to 1.0). The remaining setup that is common to every experiment:

- **Data and tokeniser.** The `nanochat` pre-training text mix (a ClimbMix derivative) tokenised with the RustBPE tokeniser, $|V| = 32{,}768$. Context length $T = 512$.
- **Compute budget.** $n_{\text{steps}} = 16{,}000$ at total batch $48 \cdot 512 = 24{,}576$ tokens per step, i.e. $\sim 3.93{\cdot}10^8$ training tokens per run. The vanilla baseline uses device batch 48 with grad_accum = 1; LUTGPT uses device batch 24 with grad_accum = 2 for memory reasons. The effective batch is identical in both cases.
- **Hardware.** Single H100 GPU per run. The vanilla baseline completes in $\sim$ 37 minutes; the full-width LUTGPT run (`exp754`) completes in $\sim$ 6.3 hours; the narrow-backbone LUTGPT run (`exp755`) completes in $\sim$ 5.4 hours.
- **Evaluation metric.** Validation loss is reported in *bits per byte* (bpb): the per-token cross-entropy on the held-out validation text, summed in bits over the validation tokens and divided by the total raw-byte count of the same text,

$$\text{bpb} \;=\; \frac{-\sum_i \log_2 p(t_i \mid t_{<i})}{\sum_i \text{bytes}(t_i)},$$

  where $\text{bytes}(t_i)$ is the number of UTF-8 bytes the BPE token t_i represents. Dividing by raw bytes rather than tokens makes the metric tokeniser-independent: a model with a 32,768-vocab BPE and one with a different vocabulary on the same text are directly comparable. Lower is better. We use the `evaluate_bpb` routine from `nanochat`, averaged over `eval_steps` = 10 micro-batches at the same context length as training.

6.2 Vanilla baseline (`exp709`)

The vanilla reference is the untied `MinimalGPT+RoPE` architecture of Section 3.1: pre-norm transformer block (`LayerNorm` $\to$ SDPA attention $\to$ residual; `LayerNorm` $\to$ GELU MLP $\to$ residual), residual width $D = 384$, depth $L = 6$, $H = 6$ heads with per-head dimension 64, FFN hidden $= 4D$, untied unembedder. All Linear and Embedding weights are initialised from $\mathcal{N}(0, 0.02^2)$; the attention output projection and the second MLP linear are zero-initialised, so every residual branch starts at zero and the model begins as the identity map on the embedding.

At 35.79 M parameters this is the standard nanoGPT-class point of comparison and our headline number to beat (or land near). The final validation loss after 16,000 steps is

`exp709`: **val_bpb** = 1.1922 at 35.79 M parameters.

6.3 LUTGPT, full-width backbone (`exp754`)

The headline LUTGPT configuration uses the architecture of Section 3.2 at full backbone width.

- $E = D = 384$, $L = 6$, $H = 6$, $d_{qk} = d_v = 64$.
- Anchor-pairs per table (Section 3.7): `qk_lut` N=4 at tph=256; `v_lut` N=6 at tph=256; `out_proj` N=7 at tph=512; `residual_lut` and `emb_resid_lut` N=6 at tph=256.
- Forward mode `hybrid_smooth` with backward mode `dense_K`, held fixed for all 16,000 steps.

`exp754`: **val_bpb** = 1.1842 at 276.8 M parameters.

This is 8.0 mb below the vanilla baseline at 7.7× the parameter count. We are not claiming a parameter-efficient win; the LUT family carries more parameters by construction (a $K \cdot \text{tph}$ table per output channel against a single dense weight matrix). What this result establishes is that at matched data and matched training-token budget, the LUT family lands on the same side of the vanilla curve — a result we did not have access to before this run.

6.4 LUTGPT, narrow backbone (`exp755`)

A second LUTGPT configuration narrows the backbone — the rank-coded stream where the LUTs actually compute — to half-width, keeping the Euclidean D-stream wide for the unembedder read-out.

- $E = 192$ (backbone), $D = 384$ (D-stream accumulator), $L = 6$, $H = 6$, $d_{qk} = 64$, $d_v = 32$.
- All other settings identical to `exp754` (anchor-pair budget, forward/backward modes, training recipe).

`exp755`: **val_bpb** = 1.2044 at 176.2 M parameters.

This is 12.2 mb above the vanilla baseline at 4.9× the parameter count. The point of this run is structural rather than headline: the LUT-bearing backbone can be halved in width with only a ~ 20 mb loss penalty relative to the full-width `exp754`, while the wide D-stream accumulator (read only by the linear unembedder) is preserved unchanged. This validates the asymmetric architecture of Section 3.4: most of the LUT compute belongs in a narrow rank-coded stream, and only the unembedder needs a wide Euclidean input.

6.5 The soft-to-hard gap

Both `exp754` and `exp755` are trained with the hybrid-smooth forward (Section 2.2). Hybrid-smooth eval is the in-distribution measurement of a hybrid-smooth-trained model; hard eval is the measurement under the magnitude-blind forward that maps directly to rank-coded sparse-lookup hardware. The gap between them is the structural cost of hardware-targetedness.

We measure two things: (i) the soft-to-hard gap on the same trained checkpoint, and (ii) the alternative of training in hard mode from scratch.

run	params	trained mode	eval mode	val_bpb
exp754	276.8 M	hybrid_smooth	hybrid_smooth	1.1842
exp754	276.8 M	hybrid_smooth	hard	1.2498
exp752	276.8 M	hard	hard	1.2162
exp755	176.2 M	hybrid_smooth	hybrid_smooth	1.2044
exp755	176.2 M	hybrid_smooth	hard	1.2778
exp756	176.2 M	hard	hard	1.2301

Two observations.

The soft-trained model loses 65–73 mb when forced into hard forward at eval time. exp754 drops from 1.1842 to 1.2498 (+65.6 mb) when its forward is switched to hard at eval; exp755 drops from 1.2044 to 1.2778 (+73.4 mb). The hybrid-smooth-trained model has learnt to lean on the small-magnitude top-k blending that the hard forward discards. The soft-to-hard pipeline at eval time is not free.

Training in hard mode from scratch closes most of that gap. exp752 (same architecture as exp754, trained *in* hard mode) lands at 1.2162 — 33.6 mb below exp754's hard-eval number. exp756 (same architecture as exp755, hard-trained) lands at 1.2301 — 47.7 mb below exp755's hard-eval number. The deployment-quality model is the hard-trained one; soft training is not a free curriculum towards a hard-deployable model.

The takeaway is that the choice of forward mode at training time should match the deployment target. Hybrid-smooth is the right choice when the model will be evaluated in hybrid-smooth, and a hard-trained model is the right choice for deployment on rank-coded hardware — with the loss numbers under the matched-to-vanilla framing being exp752 at 1.2162 for the full-width backbone and exp756 at 1.2301 for the narrow backbone.

6.6 Wall-clock-matched hard training

*This subsection is a placeholder; the run **exp760** is currently in progress on a parallel machine. **exp760** spends **exp755**'s hybrid-smooth wall-clock budget on hard training at the same $E = 192$ narrow-backbone architecture; if the prediction holds, **exp760** should land below **exp756**'s* 1.2301*, demonstrating that the wall-clock cost of hybrid-smooth training is better spent on additional hard-training steps when the deployment target is hard.*

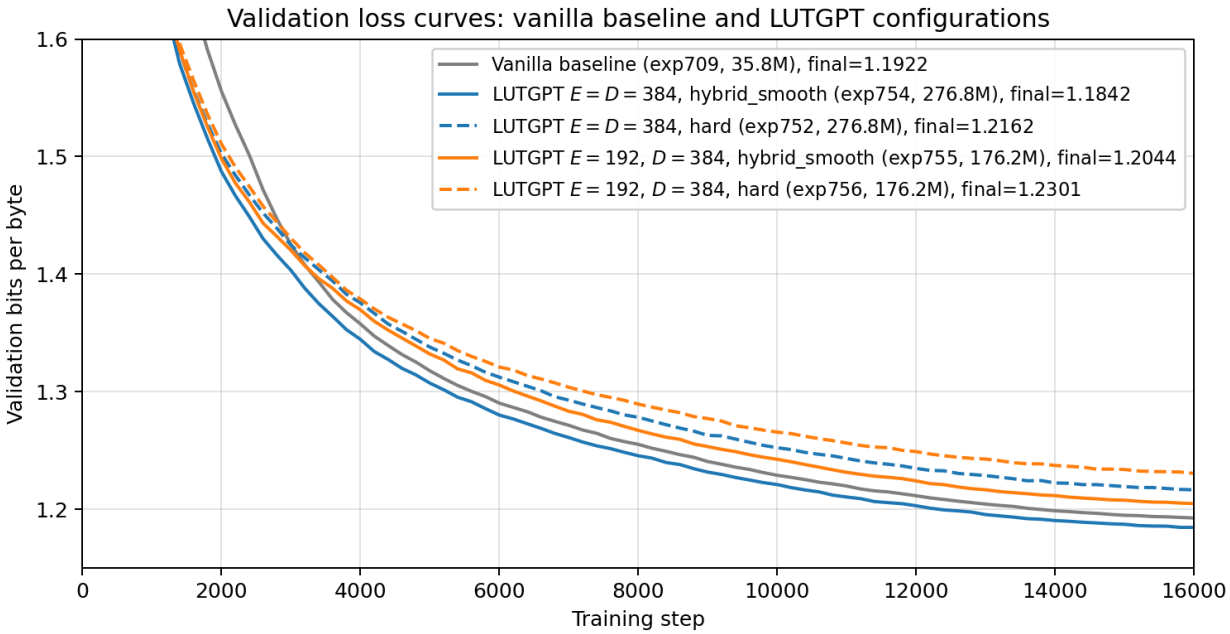

Figure 3: Validation bits-per-byte vs. training step for the five runs reported in Section 6. Solid lines: hybrid-smooth-trained models; dashed lines: hard-trained models. Vanilla baseline (`exp709`) in grey, full-width LUTGPT pair (`exp754` hybrid-smooth, `exp752` hard) in blue, narrow-backbone LUTGPT pair (`exp755` hybrid-smooth, `exp756` hard) in orange. Eval-mode-only measurements from Section 6.5 (the soft-trained models scored under hard forward) are not curves and appear only in the table above.

7 Open problems

Two residual matmul interfaces remain in LUTGPT, and both are open research problems whose resolution would close the loop to end-to-end LUT-class hardware execution.

LUT-based attention. $\texttt{softmax}(Q \cdot K^\top) \cdot V$ is the one operation in the backbone that we did not replace with a LUT-class primitive. The difficulty is reproducing softmax's data-dependent resolution — focusing sharply on a few keys when the match is confident, spreading broadly when it is uncertain — inside a rank-coded primitive. Pure sign-pack lookups cannot produce a continuous attention distribution; the LUT family would need an additional mechanism (a learnt mixing kernel, a sparse top-k readout, or something else) for this. We do not solve this here.

LUT-based unembedder. The linear layer $\mathbb{R}^D \to \mathbb{R}^{|V|}$ producing logits is the second matmul interface. The difficulty is generating magnitude-coded logits over a 32,768-way vocabulary from a rank-coded representation, in a way that preserves the gradient signal a cross-entropy loss expects. We have no candidate replacement that we are confident about; the D-stream architecture exists precisely because the unembedder needs Euclidean input.

Resolving either problem would close the gap independently of the other; resolving both would make LUTGPT end-to-end LUT-class, and the hardware story above becomes a complete description of the model rather than a description of its backbone.

8 Conclusion

We have presented LUTGPT, a transformer-class model in which every attention projection and every residual contribution is a differentiable lookup table, with a soft-surrogate backward, two forward modes, and a published architecture that matches a vanilla, RoPE-augmented baseline with a separate unembedder on the nanochat data, with the comparison hyperparameters matched on both sides. On current GPU hardware LUTGPT is not competitive in wall-clock; on the abstract level of backbone bandwidth-per-token and active-parameter-per-token it shows a $\sim$20$\times$ higher compression density than the matched baseline. We have been explicit about which matmul interfaces remain — SDPA and the linear unembedder — and treat both as open problems. The contribution of this report is two-fold: the architectural existence proof that the rank-coded LUT family extends to a real text-data model at meaningful scale, and the explicit identification of the architectural properties that would make this family the preferred choice on specialised rank-coded sparse-lookup hardware.

A Implementation notes

The code accompanying this report is the `spiky` repository at `https://github.com/anatoli-starostin/spiky`, on the `publish/lutgpt` branch. The LUT primitive of Section 2 lives in `src/spiky/lutorch/fast_multi_head_lut.py` as `FastMultiHeadLut`.

Two end-to-end entry points reproduce the report's models at different scales:

- `examples/lutgpt/` ships the narrow-backbone reference configuration of Section 6.4 ($E = 192$, $D = 384$, hybrid-smooth forward for 16,000 steps; the published `exp755` recipe). `train.py` reads `config.json` and builds the model of Section 3.2. Requires a local checkout of `nanochat` [2] pointed to by the `NANOCHAT_ROOT` environment variable for the tokeniser, dataloader, and bpb evaluator.
- The single-GPU walkthrough `workbooks/nanochat_walkthrough.ipynb` loads the tokenised dataset, trains the vanilla `MinimalGPT+RoPE` baseline of Section 3.1, then trains a LUTGPT at the full-width architecture of Section 6.3 (`exp754`; $E = D = 384$, $d_v = 64$) at a reduced budget (device batch 8, 8,000 steps) so the whole run fits in a few hours.

References

[1] E. Izhikevich. The spiking manifesto: ranking-coded computation and differentiable lookup tables. Technical report, 2025. [internal reference; placeholder citation]

[2] A. Karpathy. nanochat. `https://github.com/karpathy/nanochat`, 2025.

[3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, I. Polosukhin. Attention is all you need. *NeurIPS*, 2017.

[4] J. Su, Y. Lu, S. Pan, B. Wen, Y. Liu. RoFormer: enhanced transformer with rotary position embedding. *arXiv:2104.09864*, 2021.

[5] X. Chen, C. Liang, D. Huang, E. Real, K. Wang, Y. Liu, H. Pham, X. Dong, T. Luong, C.-J. Hsieh, Y. Lu, Q. V. Le. Symbolic discovery of optimization algorithms. *NeurIPS*, 2023.

[6] I. Loshchilov, F. Hutter. Decoupled weight decay regularization. *ICLR*, 2017.